School Basics

Full Stack Software Engineer — Practical Technical Assessment

Submission deadline: within 48 hours of receiving the assessment

Purpose

This exercise helps us understand how you approach a practical software problem, structure maintainable Python code, handle API constraints and failures, work with data, and test your solution. We value clear reasoning and sound trade-offs more than a large or polished application.

Scenario

School Basics needs to import records from an external service into an internal system. The external service has a daily request limit. Requests may fail temporarily, and the import should avoid creating duplicate records when retried.

Assumptions for this exercise

- The external service allows at most 100 requests per day.
- Each request can retrieve up to 25 records.
- A request may fail with a temporary error or return HTTP 429 (rate limit exceeded).
- Each record has a unique external_id and a name.
- You do not need to call a real API or connect to a real database.

Your task

1. Import logic — Write a Python function that accepts a list of records and processes them in batches of up to 25. Explain how your approach respects the daily request limit. You may implement a simplified version rather than a complete production service.

2. API failures and retries — Show how you would handle a temporary request failure and HTTP 429. Explain when you would retry, how you would avoid retrying forever, and how you would use a delay/backoff strategy. If you do not implement actual waiting, make the behavior clear in code or comments.

3. Data persistence — Show how you would store or upsert records using SQL, or provide a small database-layer function/pseudocode with a concrete SQL statement. Explain how you would avoid duplicates using external_id.

4. Automated test — Write at least one PyTest test for normal behavior and, if time allows, a test for a failure or retry case. Mock external API calls; do not make real network requests.

5. Deployment and maintainability — In a short note, identify two things you would add before deploying this to production—for example, logging, configuration/secrets handling, monitoring, transaction handling, or a safe way to resume an interrupted import.

Starter data

Use these sample records for your example and tests:

external_id	name
SB-1001	Notebook
SB-1002	Pencil Case
SB-1003	Backpack
SB-1004	Folder

Deliverables

- Python source file(s), or a clearly organized code snippet.
- At least one PyTest test file.
- A brief README or note explaining how to run the code/tests and any assumptions.
- A short explanation of your retry and duplicate-prevention approach.

Tools and expectations

- You may use official documentation and AI tools. Please disclose briefly which tools you used and what you used them for.
- You should be able to explain and defend the code you submit during a short follow-up discussion.
- Do not include credentials, private company information, or code from a current or former employer.
- Do not spend time building a user interface or deploying a live service.

Submission

Reply to the email that provided this assessment and attach your files or provide a link to a repository/document that the hiring team can access. Please submit within 48 hours. We are assessing your approach, not how much extra time you spend.